Ahsanullah University of Science and Technology



Department of Electrical & Electronic Engineering

OPEN ENDED LAB

Course No : **EEE 4106**

Course Name : **Control Systems Laboratory**

Date of Submission : 29 /09/ 2026

Project Name : **DESIGN AND IMPLEMENTATION OF A 2-DOF RECORD-AND-PLAYBACK ROBOTIC ARM WITH CLOSED-LOOP JOINT POSITION CONTROL**

Submitted By:

ID : 20220205048
: 20220205050
: 20220205056
: 20220205057
: 20210105155

Section : A

Year : 4th

Semester : 1st

Introduction

This project presents the design and implementation of a two-degree-of-freedom (2-DOF) robotic arm that can be taught a sequence of joint positions by hand and then repeat that sequence automatically. Each joint is driven by a 25GA DC gear motor through an L298N H-bridge driver, and the angle of every joint is measured by an AS5600 12-bit contactless magnetic encoder. An Arduino Uno runs a separate closed-loop position controller for each joint. In teach mode the motors are disabled, the arm is moved manually and the two joint angles are stored as waypoints each time the Record button is pressed. In playback mode the stored angles become reference inputs, and the controller continuously compares them with the measured angles and drives the motors to reduce the error to within a small tolerance before moving to the next waypoint. The report models the joint as an armature-controlled DC motor, derives the closed-loop transfer function, explains the effect of the proportional, integral and derivative terms on transient response and steady-state error, and discusses how the controller was implemented in discrete time on the microcontroller. Practical issues such as gravity loading, gearbox backlash, motor dead-zone and angle wrap-around are identified, and the measured performance of the system is reported and analysed. Industrial robots used for welding, painting, packaging and pick-and-place are commonly programmed by a method called "teach and repeat" (lead-through programming). An operator moves the robot through the required motion, the controller records the joint positions, and the robot later reproduces the motion without human involvement. What makes this possible is not the mechanical structure alone but the feedback control loop at each joint: the robot must know where each joint actually is and must correct any difference between the actual and the desired position.

This project applies the same principle on a small scale. A 2-DOF arm was built using steel links, two 25GA gear motors and two AS5600 magnetic encoders, all coordinated by an Arduino Uno. The system is a closed-loop control system in the strict sense required by the course: it senses the output (joint angle), computes the error with respect to a reference (the recorded angle), and actuates the motor in a direction and with a magnitude that reduces that error. Without the encoder feedback the arm could only run the motors for a fixed time, and the final position would change with battery voltage, load and friction. With feedback, the arm returns to the same angles regardless of these variations, within the limits of the hardware.

2. Objectives

1. To design and build a 2-DOF robotic arm in which each joint is actuated by a DC gear motor and sensed by an absolute magnetic angle sensor.
2. To implement a closed-loop position controller (PID) for each joint on an Arduino Uno, using AS5600 encoder feedback.

3. To implement a teach mode in which joint angles are recorded as waypoints while the arm is moved by hand, and a playback mode in which the arm repeats the recorded sequence automatically.
4. To model the joint dynamics, derive the closed-loop transfer function and relate the controller gains to stability, transient response and steady-state error.
5. To evaluate the system experimentally in terms of rise time, overshoot, settling time, steady-state error and repeatability, and to discuss its limitations.

3. Theoretical Background

3.1 Open-loop versus closed-loop control

In an open-loop system the controller output does not depend on the actual output. For a DC motor this would mean applying a voltage for a fixed time and hoping the joint reaches the required angle. Any change in load, supply voltage or friction then produces a different final position, and there is no mechanism to correct it. In a closed-loop (feedback) system the output is measured and subtracted from the reference to form an error signal $e(t) = \theta ref(t) - \theta(t)$. The controller acts on this error, so disturbances and parameter variations are automatically compensated as long as they are within the actuator's capability. The arm in this project is a closed-loop system: the AS5600 provides the feedback path, the Arduino forms the error and computes the control law, and the L298N and motor form the actuator.

3.2 Model of a joint (armature-controlled DC motor)

Each joint consists of a permanent-magnet DC motor, a reduction gearbox and the arm link. The electrical and mechanical equations of the motor are:

$$Va(t) = Ra \cdot ia(t) + La \cdot dia/dt + Kb \cdot \omega m(t) \quad (1)$$

$$Tm(t) = Kt \cdot ia(t) \quad (2)$$

$$J \cdot d\omega m/dt + B \cdot \omega m(t) = Tm(t) - TL(t) \quad (3)$$

where Va is the armature voltage, Ra and La are the armature resistance and inductance, Kb is the back-EMF constant, Kt is the torque constant, J and B are the total inertia and viscous friction referred to the motor shaft, and TL is the load torque (mainly gravity acting on the link). Taking the Laplace transform with zero initial conditions and TL = 0, the transfer function from armature voltage to motor shaft angle is:

$$\Theta m(s) / Va(s) = Kt / \{ s \cdot [(J \cdot s + B)(La \cdot s + Ra) + Kt \cdot Kb] \} \quad (4)$$

For small DC motors the electrical time constant La/Ra is much smaller than the mechanical time constant, so La can be neglected. Including the gear ratio N ($\theta = \theta m / N$), the joint can then be represented by the well-known second-order model:

$$G(s) = \Theta(s) / Va(s) = K / [s \cdot (\tau \cdot s + 1)] \quad (5)$$

$$K = Kt / [N \cdot (Ra \cdot B + Kt \cdot Kb)] , \quad \tau = Ra \cdot J / (Ra \cdot B + Kt \cdot Kb) \quad (6)$$

The free integrator (the factor 1/s) exists because a motor converts voltage into speed, and angle is the integral of speed. This makes the plant a Type-1 system, which has an important consequence for steady-state error discussed in Section 3.4.

3.3 Closed-loop block diagram

Figure 1 shows the control loop used for each joint. The two joints use identical but independent loops (decentralised control). The L298N is modelled as a gain Kd = Vs/4095 that converts the 12-bit PWM command into an average armature voltage, and the AS5600 is modelled as a unity-gain sensor because its 12-bit output is converted directly to degrees in software (θ = raw × 360 / 4096). The load torque Td caused by gravity and friction enters the loop as a disturbance at the motor.

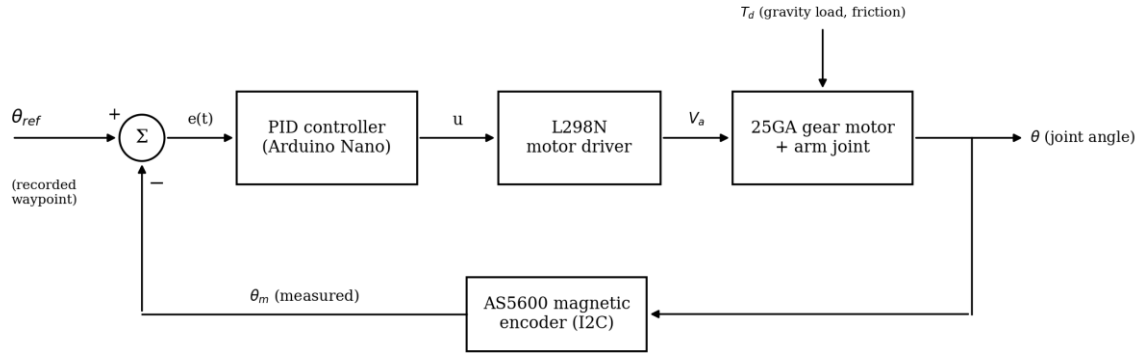


Figure 1: Closed-loop position control block diagram of one joint

3.4 Proportional control and the effect of gain

With a proportional controller C(s) = Kp, the open-loop transfer function is L(s) = K′ / [s(τs + 1)] where K′ = Kp·Kd·K. The closed-loop transfer function with unity feedback is:

$$T(s) = \Theta(s)/\Theta ref(s) = (K'/\tau) / [\, s^2 + (1/\tau)\cdot s + K'/\tau \,] \quad (7)$$

Comparing with the standard form $\omega n^2 / (s^2 + 2\zeta\omega n\cdot s + \omega n^2)$ gives:

$$\omega n = \sqrt{(K'/\tau)}\ , \quad \zeta = 1 / (\, 2\cdot\sqrt{(\tau\cdot K')}\,) \quad (8)$$

The characteristic equation $\tau s^2 + s + K' = 0$ has all coefficients positive for any Kp > 0, so by the Routh–Hurwitz criterion the idealised loop is stable for every positive gain. Increasing Kp increases ωn (faster response) but reduces ζ, which increases the percentage overshoot, given by $\%OS = 100\cdot\exp(-\zeta\pi / \sqrt{(1-\zeta^2)})$. The settling time is approximately $t_s \approx 4 / (\zeta\omega n) = 8\tau$, which for the pure P case is independent of Kp once the response is underdamped. In the real system, PWM saturation, the sampling delay of the digital controller and gearbox backlash add phase lag that is not present in Equation (7), so a very high Kp produces sustained oscillation (hunting) around the target. This is why the gain cannot simply be made as large as possible.

Because the plant is Type-1, the steady-state error to a step reference is zero in the ideal model. However, a constant disturbance torque such as gravity on the shoulder joint produces a non-

zero steady-state error with P-only control: the motor needs a non-zero voltage to hold the link, and a P controller can only produce a non-zero output when the error is non-zero. The motor dead-zone (the minimum PWM needed to overcome static friction) has a similar effect.

3.5 PID control

To address the limitations above, a PID law is used:

$$C(s) = Kp + Ki / s + Kd \cdot s \tag{9}$$

The integral term accumulates error over time and keeps increasing the output until the error is removed, which eliminates the steady-state error caused by gravity and friction (the system becomes Type-2 with respect to the reference). The derivative term reacts to the rate of change of the error and adds damping, reducing overshoot caused by a high Kp. The trade-off is that too much integral gain slows the response and causes overshoot, and too much derivative gain amplifies sensor noise and quantisation steps.

3.6 Discrete-time implementation

The controller runs on the Arduino in a fixed-period loop with sample time T. Using a rectangular approximation of the integral and a backward difference for the derivative, the control law becomes:

$$u[k] = Kp \cdot e[k] + Ki \cdot T \cdot \Sigma\, e[j] - Kd \cdot (\theta[k] - \theta[k-1]) / T \tag{10}$$

The derivative is taken on the measured angle instead of the error so that a sudden change of reference (moving to the next waypoint) does not create a large spike in the output ("derivative kick"). The output u[k] is then limited to ±4095; its sign sets the direction pins of the L298N and its magnitude sets the PWM duty cycle. The integral sum is clamped (anti-windup) so that it does not grow without limit while the output is saturated. The sample time must be short compared with the mechanical time constant τ; otherwise the delay introduced by sampling reduces the phase margin and the loop becomes oscillatory.

4. System Design and Methodology

4.1 Overall working principle

The arm has two operating modes selected with two push buttons. In **teach (record) mode** the motor drivers are disabled so that the arm can be moved by hand. Each press of the Record button reads the two encoders and stores the two angles as one waypoint in memory. In **playback mode**, started by the Play button, the stored waypoints are used one after another as reference inputs. For each waypoint, all two joint controllers run simultaneously until every joint error is within a set tolerance, after which the next waypoint is loaded. When the last waypoint is reached the motors are stopped. The complete program flow is shown in Figure 2.

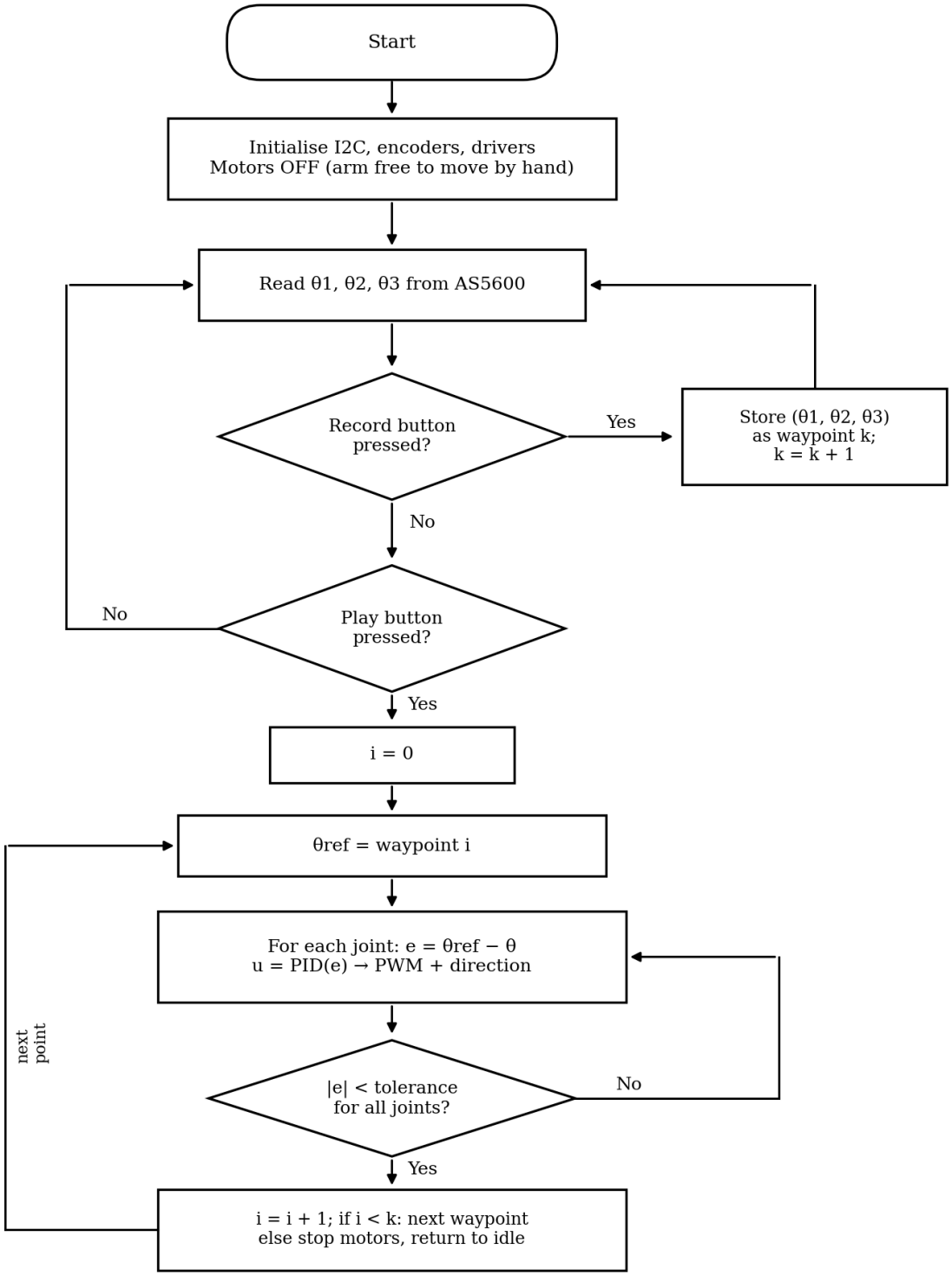


Figure 2: Flowchart of the record-and-playback program

4.2 Components and design justification

Table 1 lists the components used and the reason each was selected.

Table 1: Component list and justification

Component	Qty	Reason for selection
Arduino Uno (ATmega328P, 16 MHz)	1	Small size fits on the arm base; has hardware I2C (A4/A5) for the encoders and six PWM pins, enough for two motor channels; fast enough to run two control loops at a sample time of a few milliseconds.
25GA DC gear motor	2	High output torque at low speed due to the built-in gearbox, which is required to hold the steel links against gravity; compact metal gearbox; 12 V rating matches the 3-cell battery.
L298N dual H-bridge driver	1	Allows bidirectional control (needed to correct error in both directions) and PWM speed control; each module drives two motors, so two modules cover two joints; on-board 5 V regulator supplies the logic.
AS5600 magnetic encoder	2	Absolute 12-bit angle (4096 counts per revolution, about 0.088°), so the position is known at power-up without homing; contactless, so no wear; digital I2C output is less sensitive to noise than an analogue potentiometer.
Push button	6	Record and Play commands; wired to digital inputs with internal pull-up resistors, so no external resistor is required.
3 × Li-ion cell (3S) with holder	1 set	About 11.1 V nominal (12.6 V fully charged), suitable for 12 V motors after the L298N voltage drop; high current capability; rechargeable.
Steel arm links	3	Rigid, so the links do not bend and the encoder angle represents the true link position.
Motor mount and 4 mm coupler	2 each	Fix the motors to the frame and connect the motor shaft to the joint, keeping the shaft and the encoder magnet aligned.
Buzzer	1	Gives sounds when clicking record button

4.3 Mechanical design

The mechanical structure was first modelled to check the link lengths, motor positions and clearances before fabrication (Figure 3). Each joint has a motor fixed on a mount and coupled to the next link with a 4 mm coupler. A diametrically magnetised magnet is fixed to the rotating side of each joint and the AS5600 is placed facing it on the stationary side, so the sensor measures the joint angle directly at the output, after the gearbox. Measuring at the output rather than at the motor shaft means gearbox backlash does not hide the true link position from the controller.

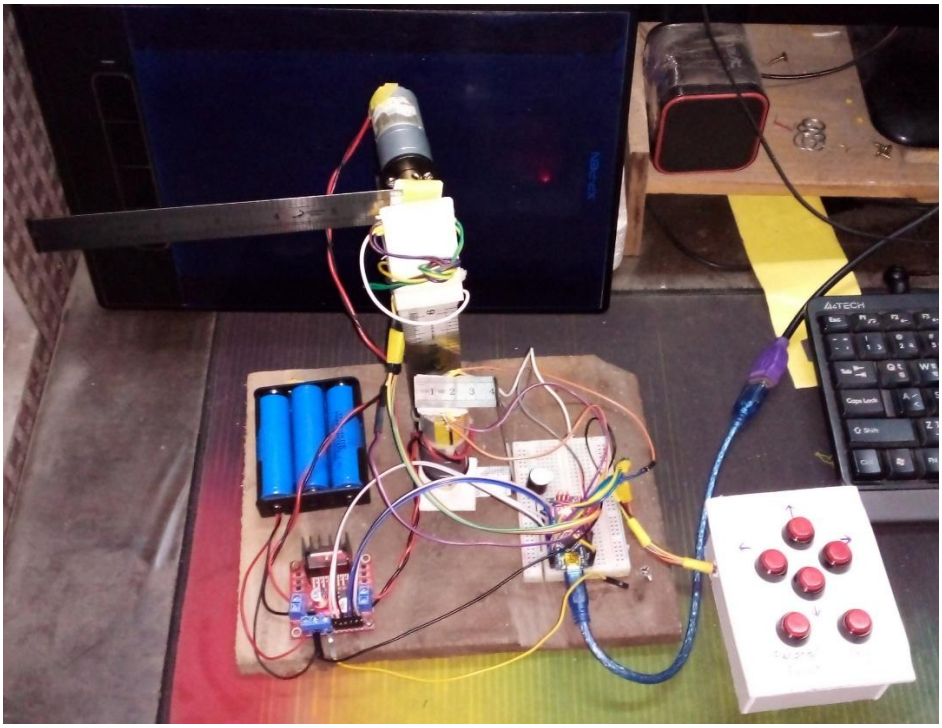


Figure 3: Design of the arm

4.4 Electrical design

The complete circuit is shown in Figure 4 and the pin assignment in Table 2. The 3S battery pack supplies the motor input terminals of both L298N modules. The 5 V output of the L298N on-board regulator supplies the Arduino Uno and the two AS5600 sensors, and all grounds are connected together so that the logic signals share a common reference. The two push buttons connect between analogue pins A2/A3 (used as digital inputs) and ground.

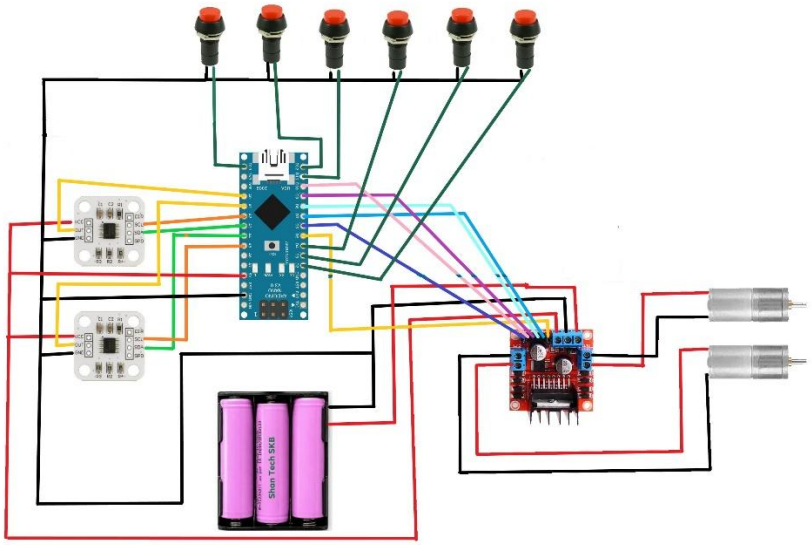


Figure 4: Circuit diagram of the system

Note on the encoder bus: Every AS5600 has the same fixed I2C address (0x36), so two sensors connected in parallel to A4/A5 cannot be read individually. **[Group to state the method used in the final hardware, e.g. a TCA9548A I2C multiplexer, separate software I2C pins for two of the sensors, or reading the analogue OUT pin of each sensor on A0, A1 and A6.]**

4.5 Software implementation

The program was written in the Arduino IDE (C++). Its main parts are:

- **Angle reading and calibration:** The 12-bit raw angle of each sensor is converted to degrees. At start-up the angle of each joint in its rest position is saved as an offset, so that the working angle is measured from a known zero.
- **Wrap-around handling:** Since the sensor output jumps from 359.9° to 0°, the error is always calculated as the shortest angular difference, limited to the range −180° to +180°. Without this, a joint near the jump point would rotate almost a full turn in the wrong direction.
- **PID with practical limits:** The output is saturated to ±255, the integral term is clamped (anti-windup), a minimum PWM is applied when the output is non-zero to overcome the motor dead-zone, and a small dead-band around zero error stops the motor from chattering once the joint is on target.
- **Waypoint storage:** Each waypoint is two integers (6 bytes), stored in an array in RAM. A waypoint counter limits the number of points so the 2 KB SRAM of the Uno is not exceeded.
- **Button handling:** Buttons use INPUT_PULLUP and a software debounce delay so that one press records exactly one waypoint.
- **Waypoint arrival:** A waypoint is considered reached when the error of all two joints stays within the tolerance for several consecutive samples. A time-out prevents the program from waiting forever if one joint cannot reach its target (for example, because of an obstruction).

4.6 Controller tuning

The gains were tuned experimentally for one joint at a time with the other joints held still. Kp was increased from a small value until the joint responded quickly with slight overshoot; Kd was then increased to damp the overshoot; finally a small Ki was added until the remaining steady-state error due to gravity disappeared. The angle was printed on the Serial Plotter during tuning so that the step response could be observed directly. Joints that carry more load (the base and shoulder) generally need a larger integral gain than the lighter distal joint. The final values are listed in Table 3.

Table 3: Final controller parameters

Joint	Kp	Ki	Kd	Tolerance (°)	Sample time T (ms)
Joint 1 (base)					
Joint 2 (shoulder)					

5. Results

5.1 Step response of each joint

Each joint was commanded to move by a fixed step while the angle was logged through the serial port. Rise time (10 % to 90 %), percentage overshoot, 2 % settling time and steady-state error were measured from the logged data. The results are given in Table 4 and a typical response is shown in Figure 5.

Table 4: Step response of each joint

Joint	Step (°)	Rise time (s)	Overshoot (%)	Settling time (s)	Steady-state error (°)
Joint 1					
Joint 2					

6. Discussion

The results show that feedback allows the arm to return to recorded positions regardless of the load and the battery voltage, which would not be possible with timed open-loop motor control. The following factors limit the accuracy of the system and explain the remaining errors.

- **Gravity loading:** the base/shoulder joint carries the weight of the rest of the arm, and the torque required depends on the arm configuration. With P-only control this shows up as a steady-state offset that changes with the pose of the arm. The integral term removed this offset, but a large Ki caused overshoot and slow oscillation, so a compromise value has to be chosen.
- **Dead-zone and static friction:** below a certain PWM value the gear motor does not rotate at all. Small errors therefore produce no motion until the integral builds up. The minimum-PWM compensation reduces this effect but, if set too high, makes the joint oscillate around the target. The dead-band tolerance was chosen to avoid this.
- **Backlash:** the gearbox has some free play. Because the encoder is on the output side, the controller sees the true link angle, but backlash introduces a non-linearity (the motor turns a small amount before the link moves) that limits how tightly the loop can be tuned.
- **Driver losses:** the L298N uses bipolar transistors and drops roughly 2 V or more at load, so the motors receive less than the battery voltage and the driver heats up. A MOSFET driver would be more efficient.

- **Independent joint control:** each joint is controlled separately and coupling between joints (motion of one joint changing the load on another) is treated as a disturbance. This is acceptable at the low speeds used, but would limit performance at higher speeds.
- **Motion between waypoints:** the arm moves to each waypoint as a step change in reference, so joints arrive at different times and the path between waypoints is not controlled. Generating a smooth trajectory (for example, linear interpolation between waypoints) would give smoother, more natural movement.

Possible improvements include using a MOSFET-based motor driver, adding a small trajectory generator between waypoints, storing waypoints in EEPROM so that the sequence is kept after power-off, adding gravity feed-forward compensation to the shoulder joint, and adding a gripper as an end effector.

7. Conclusion

A 2-DOF record-and-playback robotic arm was designed, built and tested. Each joint forms a closed-loop position control system consisting of an AS5600 magnetic encoder for feedback, an Arduino Uno running a discrete PID controller, and an L298N driver with a 25GA gear motor as the actuator. The joint was modelled as a Type-1 second-order system, and the model was used to explain the influence of the controller gains on speed, overshoot and steady-state error, in particular why integral action is needed to overcome gravity and friction. In operation, the arm can be taught a sequence of positions by hand and can repeat that sequence automatically using the recorded angles as reference inputs. The project demonstrates in practical form the main ideas of feedback control: error correction, the trade-off between speed and stability, and rejection of disturbances.

8. Individual Contribution

Student ID	Contribution
20220205048	Arduino programming of record and testing
20220205050	Problem definition, mechanical design, fabrication and assembly of the arm structure.
20220205056	Component selection, circuit design, wiring of the motor drivers, encoders and power supply.
20220205057	Mathematical modelling of the joint, PID controller design and tuning.
20210105155	Data collection and result analysis.

References

[1] N. S. Nise, Control Systems Engineering, 7th ed. Hoboken, NJ, USA: Wiley, 2015.

[2] K. Ogata, Modern Control Engineering, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.

[3] K. J. Åström and T. Hägglund, PID Controllers: Theory, Design, and Tuning, 2nd ed. Research Triangle Park, NC, USA: ISA, 1995.

[4] J. J. Craig, Introduction to Robotics: Mechanics and Control, 3rd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2005.

[5] ams AG, "AS5600 – 12-bit programmable contactless potentiometer," datasheet.

[6] STMicroelectronics, "L298 – Dual full-bridge driver," datasheet.

[7] Microchip Technology, "ATmega328P 8-bit AVR microcontroller," datasheet.

[8] Arduino, "Arduino Uno – product documentation," docs.arduino.cc.